

Lab 3: Wrangling data with dplyr

! Important

This lab is due on Friday, Sept 4 at 11:59pm. To be considered on time, the following must be done by the due date:

- Final `.pdf` file submitted on Gradescope

Be sure to **assign questions to pages** when you turn in your assignment on Gradescope. A .5 point deduction will be made any time the question is not assigned to a page within your PDF.

- Select the first page of your .PDF submission to be associated with the “Formatting” section.

Set up

1. In Canvas, you'll find the template for this assignment (“Lab 03 Template”). Download the file and open it.
2. Save it in your class project folder as “`lab-03.qmd`”.
3. Open the document. Replace “Your Name Here” with your name in the YAML (the header of the document). Also, replace “Today’s Date Here” with today’s date.

Wrangling Data

Today’s lab will focus on data wrangling using the `tidyverse` (in particular, basic functions in `dplyr`). The lab has a pretty hefty guided portion – follow along with the code that is written to familiarize yourself with the commands. You are highly encouraged to ask your lab TA or each other for help!

The data are available by typing in the following. Note that we are naming the dataset `ncbikecrash` - you may name the dataset something else if you wish, but make sure that you are using this name consistently throughout the lab!

```
ncbikecrash <- read.csv("https://karamccor.github.io/b6002/labs/data/bikecrash.csv")
```

Since we are using the `tidyverse` package, we can load the package in the Console.

```
library(tidyverse)
```

Warning: package 'ggplot2' was built under R version 4.4.3

— Attaching core tidyverse packages — tidyverse 2.0.0 —

✓ dplyr	1.1.4	✓ readr	2.1.5
✓ forcats	1.0.0	✓ stringr	1.5.1
✓ ggplot2	4.0.2	✓ tibble	3.2.1
✓ lubridate	1.9.3	✓ tidyr	1.3.1
✓ purrr	1.0.2		

— Conflicts — tidyverse_conflicts() —

- ✗ dplyr::filter() masks stats::filter()
- ✗ dplyr::lag() masks stats::lag()
- Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

You'll notice some “conflicts” above - this is fine. It just means that multiple packages have R functions with the same name. R will assume we want to use the functions from the `tidyverse` as opposed to the other package, in this case `stats`. `dplyr::filter()` masks `stats::filter()` means that the `filter` function from the `dplyr` package (part of the tidyverse) will be used in lieu of the `filter` function from the `stats` package. Nothing to worry about for now. (And remember that you'll **suppress these messages** in your own lab document with `#| warning: false` and `#| message: false`).

NC bike crash data

This week's data come from the North Carolina Department of Transportation. Each year, about 1,000 bicyclists are involved in police-reported crashes with motor vehicles in North Carolina, with 60 serious injuries and 20 deaths. This is a tragic yet preventable public health issue. Today's dataset provides all 7,467 police-reported bicycle crashes involving a motor vehicle in North Carolina from 2007 through 2014.

Examining our dataset

Remember, the R Console and the Quarto document live in **separate environments**. If you load the tidyverse package or the dataset in your console, it won't be loaded into the workspace for rendering the Quarto document (and vice-versa). The Console is very useful for “testing” code – seeing if what you input results in what you want. However, for creating reproducible documents, you should be editing code in your Quarto document.

Suppose we're interested in knowing the names or some basic characteristics of the `ncbikecrash` dataset. This is probably not something we want to include in our final document, but something just for our own curiosity. Try running the following two commands in the Console:

```
names(ncbikecrash)
```

```
glimpse(ncbikecrash)
```

The `names()` function returns the list of variable names in the dataset, while the `glimpse()` function (part of the `tidyverse`) provides a list of names, the type of variable (integer vs. string vs. double, etc.), as well as the first few observations of each so you get a feel for what some of the data points look like for each variable.

You can also `View` the dataset by using the `View()` function. Try running the following command in the Console:

```
View(ncbikecrash)
```

Manipulating datasets

`dplyr` is a package that allows users to do some of the most common data manipulation functions. Essentially, pliers for dataframes (hence the name). It is based on the philosophy of functions as verbs that manipulate dataframes. Under the rules of `dplyr`, the first argument is always a dataframe, subsequent arguments say what to do with that dataframe, and `dplyr`-based manipulations will always return a data frame. What are some of these verbs that say what to do with our dataframes?

- `filter`: pick rows (observations) matching certain criteria
- `slice`: pick rows (observations) using an index/indices
- `distinct`: filter for unique rows
- `select`: pick columns (variables) by name
- `arrange`: reorder rows
- `summarize`: reduce variables to summary statistic values
- `mutate`: add new variables (often used with `case_when` ...more on this later)
- ...and many more.

As you can see, many of these verbs do what you would expect them to do based on their function name.

Pipes

`dplyr` functions use a special operator, `|>`, called a **pipe**. This operator “pipes” the output of the previous line of code as the first input of the next line of code. You can think of pronouncing it as “and then.” (Note that previous versions of `tidyverse` used `%>%`, which you might come across in older documentation. `|>` is the newer version.)

Let’s use an everyday example to illustrate how a pipe works. Consider the following sequence of actions: 1) find car keys, 2) start the car, 3) drive to UNC, and finally, 4) park. if we express this sequence of actions as a set of nested functions in R “code,” this might look like

```
park(drive(start_car(find("keys")), to = "campus"))

# ...don't try running this code, it won't work.
```

If we use pipes, we have a more natural and easier to read structure:

```
find("keys") |>
  start_car() |>
  drive(to = "campus") |>
  park()

# don't try running this code, either
# still, try to "read" the code in plain English
```

Remember, the `|>` operator is pronounced “and then,” and so the above function using pipes might be pronounced find keys, and then start the car, and then drive to campus, and then park.” Much easier to figure out!

Logical operators in R

In R, there are different ways of relating variables. Consider two variables, `x` and `y`.

- `x & y`: `x` AND `y`
- `x | y`: `x` OR `y`
- `!x`: NOT `x`
- `<` and `>`: less than/greater than
- `<=` and `>=`: less than/greater than or equal to
- `==`: exactly equal to
- `!=`: not equal to
- `is.na(x)`: tests if `x` is `NA` and returns a list of `TRUE` or `FALSE` for each element of `x` (similarly, `!is.na(x)` tests for NOT being `NA`)
- `x %in% y`: tests if elements of `x` are in `y` and returns a list of `TRUE` or `FALSE` corresponding to each element of `x`

Putting it all together

In this section, we’re going to walk through some code that examines the logical operators in R. Follow along these steps that make use of some of the functions above.

filter

We use the `filter` function to select a subset of observations following some criterion(a). Suppose we are only interested in observations in Durham County. Run the following code:

```
ncbikecrash |>
  filter(county == "Durham")
```

The code should have returned a dataset with 340 rows (observations) and the same 65 variables as before. We use the logical operator `==` to take counties exactly equal to “Durham.” Remember that the output of pipe operators is another dataframe (if you are wondering what a tibble is, you can think of it as a special dataframe format specific to the `tidyverse`).

Suppose we wanted to save the Durham County crashes as a new dataframe, `durham_crashes`. We can use the assignment operator to do that:

```
durham_crashes <- ncbikecrash |>
  filter(county == "Durham")
```

Finally, we can filter for many conditions at once. Suppose we're interested in crashes in Durham County where an ambulance was required, from 2014 onward. We might use the following code:

```
ncbikecrash |>
  filter(county == "Durham" & ambulance_req == "Yes" & crash_year >= 2014)
```

We see that there are 28 such observations, and that the previous code returned a `tibble` containing their information.

slice

Instead of picking observations that match certain criteria, we can simply take a `slice` of the dataset that contains specified row numbers that we want. For instance, let's take the 4th observation:

```
ncbikecrash |>
  slice(4)
```

In `R`, the `:` specifies that we want everything from the left side of the colon to the right side. So, `1:8` would be “1 to 8.” So, if we wanted to take the first eight observations, we could take the following slice:

```
ncbikecrash |>
  slice(1:8)
```

select

On the other hand, instead of choosing rows (observations), suppose we're interested in choosing columns (variables). We can use the `select` function to do so. Say we want to keep only the `locality` and `speed_limit` variables from the dataframe. Run the following code and see what happens:

```
ncbikecrash |>
  select(locality, speed_limit)
```

We see that we have a new tibble consisting of only the `locality` and `speed_limit` variables, but for all 7,467 observations.

The magic of pipes is that we can continue to do more and more operations. Try running the following code, remembering that the pipe operator is pronounced “and then”:

```
ncbikecrash |>
  filter(county == "Durham" & ambulance_req == "Yes" & crash_year >= 2014) |>
  select(locality, speed_limit)
```

What are we doing? We are taking the original dataset `ncbikecrash`, and then filtering observations where the county was Durham AND an ambulance was required AND the crash year was 2014 or greater, and then selecting

to keep only the variables corresponding to locality and speed limit. The result is a `tibble` containing 28 observations of 2 variables.

We can also use `select` to exclude variables. Suppose we want every variable EXCEPT the “region” variable. We can easily do that by using a minus sign:

```
ncbikecrash |>
  select(-region)
```

Quick Review

What do you think the following code would do?

```
ncbikecrash |>
  filter(county %in% c("Wake", "Richmond", "Vance"))
```

How about the following code?

```
ncbikecrash |>
  filter(development == "Residential" | development == "Commercial") |>
  select(region, development, bike_age, driver_age)
```

sample_n and sample_frac

We can also take random samples from our datasets. `sample_n` takes a random sample of `n` observations, whereas `sample_frac` takes a random sample based on the fraction of observations. Both of these functions can sample with or without replacement using the option `replace = TRUE` or `FALSE`.

What is the difference between the following two sets of code?

```
ncbikecrash |>
  sample_n(5, replace = FALSE)
```

```
ncbikecrash |>
  sample_frac(0.2, replace = FALSE)
```

The first example using `sample_n` returned a `tibble` with 5 observations, while the second example using `sample_frac` returned a `tibble` with one-fifth of the total observations in the dataset (proportion equal to 0.2 out of 1). As always, you can save these as new datasets.

The following code would save a random sample of 1/3 of the `ncbikecrash` dataset as a new dataset called `ncbikecrash_sample`:

```
ncbikecrash_sample <- ncbikecrash |>
  sample_frac(1/3, replace = FALSE)
```

Notice that here we used a fraction instead of a decimal.

distinct and arrange

The `distinct` function returns all unique observations. Suppose we're interested in all cities and counties that had a bike crash. Run the following code:

```
ncbikecrash |>
  select(county, city)
```

As expected, we have a `tibble` containing every county/city combination in the dataset, and have only those variables. However, there are a lot of repeats! We can find the unique observations as follows:

```
ncbikecrash |>
  select(county, city) |>
  distinct()
```

“Take the `ncbikecrash` data, and then select county and city, and then keep only the distinct observations.”

Note that these are in the order that they first appeared in the dataset. We can `arrange` them in alphabetical order as follows:

```
ncbikecrash |>
  select(county, city) |>
  distinct() |>
  arrange(county, city)
```

The above code, after taking the distinct observations, then arranges in order by county, and then by city. If we wanted to order in the opposite direction (descending, instead of ascending), we can simply use the `desc()` function:

```
ncbikecrash |>
  select(county, city) |>
  distinct() |>
  arrange(desc(county), city)
```

We see that we are ordering first in descending order by county, and then by ascending order by city (within the counties). If we have numeric data, we can also arrange to order the data based on their numeric values. The `desc()` function works the same way.

summarise and group_by

Often times, we want to summarize values in our dataset by calculating certain summary statistics such as the

mean, median, or standard deviation, etc. We can do that by using the `summarise` or `summarize` function (dplyr was written by Hadley Wickham, who is from New Zealand, hence the British spelling).

Suppose we are interested in the mean hour at which crashes occur:

```
ncbikecrash |>
  summarize(avg_hr = mean(crash_hour))
```

Here, we're creating a variable `avg_hr` that has the mean of `crash_hour`. By running the above code, you should see that the average is 14.7, or around 2:45 PM.

Handling NA values

If you have NAs in your dataset, R might throw an error if you try to calculate the `mean()` of a variable. To remove all NAs, we could do the following:

```
ncbikecrash |>
  summarize(avg_hr = mean(crash_hour, na.rm = T))
```

Here, the `na.rm = T` is telling R to remove any NA values from the `crash_hour` variable before calculating the mean.

Summarizing based on groups

We can also summarize based on different groups. For instance, if we were interested in the mean hour at which crashes occur based on whether the crash was a hit and run, we can first `group_by` the `hit_run` variable, and then summarize the mean `crash_hour` as follows:

```
ncbikecrash |>
  group_by(hit_run) |>
  summarize(avg_hr = mean(crash_hour))
```

count

It is also easy to count observations of categorical variables. Suppose we wanted to count the number of observations in each category of the variable `driver_alcohol_drugs`. We can simply run

```
ncbikecrash |>
  count(driver_alcohol_drugs)
```

mutate and case_when

Finally, one of the most common data manipulation tasks is to create a new variable based on values of existing variables. For instance, suppose we have a dataset named `dat`, that contains measurements of mass (kg), height (m) and body temperature in fahrenheit (`fahrenheit`) for study participants. We want to make new variables for

(m), and body temperature in fahrenheit (fahrenheit) for study participants. We want to make new variables for their BMI and body temperature in celsius. We can create these variables as follows (recalling the appropriate formulas):

```
dat |>
  mutate(bmi = kg/m^2)

# Don't run this code; there is no "dat" dataset
```

```
dat |>
  mutate(celsius = (fahrenheit - 32) * 5/9)

# same.
```

In doing so, we've created the new variables `bmi` and `celsius` equal to the right side of the equals sign in the `mutate` function. Most often when you define a new variable with `mutate`, you'll want to use it later (for instance, in a visualization or in downstream analyses). In this case, you'd want to save the resulting dataframe, often by overwriting the original one. As an example:

```
dat <- dat |>
  mutate(bmi = kg/m^2)
```

In this case, we've saved over the `dat` dataframe, which now contains the new variable `bmi`.

Finally, we can also do more sophisticated data creation using `case_when`, which allows for a sequence of formulas defining new variables. For instance, let's suppose we want to create a new variable that categorizes BMI based on CDC cut-offs. Consider the following code, which utilizes the `bmi` variable we saved previously:

```
dat <- dat |>
  mutate(bmi_cat = case_when(
    bmi < 18 ~ "underweight",
    bmi >= 18 & bmi < 25 ~ "normal",
    bmi >= 25 & bmi < 30 ~ "oberweight",
    bmi > 30 ~ "obese"
  ))

# Be sure all parentheses are closed!
```

We have used the `case_when` function to create a new variable. The individual cases are separated by commas, and within each case, the condition (e.g., BMI > 30) is on the left and the outcome for the new variable (e.g., "obese") is on the right of a tilde (~). We have also saved over the original dataframe in order to keep this new variable, `bmi_cat`.

Let's do a concrete example using the `ncbikecrash` data. Suppose we want to classify whether a crash occurred on a weekend or on a weekday. We will use `case_when` in combination with the logical operator `%in%` to do so (the `c()` function tells R that there is a list of things coming up):

```
ncbikecrash <- ncbikecrash |>
  mutate(weekend = case_when(
    crash_day %in% c("Saturday", "Sunday") ~ "Weekend",
    crash_day %in% c("Monday",
                     "Tuesday",
                     "Wednesday",
                     "Thursday",
                     "Friday") ~ "Weekday"
  ))
```

Let's check our work by counting the number of observations in each of our categories:

```
ncbikecrash |>
  count(weekend)
```

```
  weekend      n
1 Weekday 5658
2 Weekend 1809
```

It looks like 1809 crashes happened on weekends and 5658 occurred on weekdays.

Your turn!

Wow, that was a lot of new material. Don't worry, the more you practice, the better you'll get. Try the following exercises using the `ncbikecrash` dataset.

Note: each of the following exercises should be done in “one go” using a series of pipes. Code should be fully shown in your final PDF for each exercise. If you are asked to “create a table,” then there is no need for additional narrative, unless specifically asked to make a conclusion based on the table.

Ensure all code is visible on PDF

In your final PDF that you turn in on Gradescope, ensure all code is visible for full credit. If lines of code are too long (left to right), they may run off the page when you convert to PDF. You should consider indenting code after commas, as shown in the Weekday `case_when` example above. Ask your TAs if you need help checking this!

Exercise 1

- Run the provided code to determine the bike crashes that occurred in **residential** development areas where the driver age group was **0 to 19**. How many such bike crashes satisfy this criteria? Provide a 1-sentence response to answer this question.

```
# run this code
ncbikecrash |>
  filter(development == "Residential") |>
  filter(driver age group == "0-19")
```

Hint

To do this, run the provided code. In the `Description` of the data frame that is printed out, you'll see `df[rows x columns]`. The number of "rows" is the number of observations that satisfy these criteria.

Note: Do not display the output of all observations satisfying these criteria. Run the code yourself to find out the answer, but your template will have `#| eval: false` as a code option which **suppresses this lengthy output**.

- b. Now, copy and paste the code below into your code chunk within your quarto template. Edit the provided code so that the dataset is filtered to "Residential" development areas, the same as part (a). Then, run the code to create a table displaying the number of bike crashes in **each age group** for residential areas.

```
# edit code below: replace blank with Residential
ncbikecrash |>
  filter(development == "_____") |>
  count(driver_age_group)
```

Exercise 2

Copy and paste the code below into your code chunk within your quarto template. Edit the code to create a table which orders the most commonly occurring estimated speed from lowest to highest. (i.e., the least commonly occurring speed will appear first in the table.) To do this, replace the blank with the `driver_est_speed` variable, then run the code. **Note:** It's okay for `NA` values to be present in your table.

```
# edit code: replace blank with driver estimated speed variable
ncbikecrash |>
  count(_____) |>
  arrange(n)
```

Exercise 3

Write code that 1) filters to crashes in Durham, Orange, or Wake counties, 2) groups by county, then 3) summarizes the mean driver age in each county.

What was the mean driver age (`driver_age`) involved in a crash in each of those three counties? Display output only for those three counties.

Hint: You can filter by multiple counties using the following line of code. Recall the `|` operator means **OR**:

```
filter(county == "Henderson" | county == "Wilmington")
```

Hint: There are some missing values for `driver_age`. You can either filter for rows where the driver age isn't missing or use the `na.rm = T` option in the `mean()` function.

Exercise 4

What is the mean hour for when a crash occurs in Durham County for each month? Create a table which displays this information, arranged from earliest to latest hour.

Exercise 5

Create a new variable named `time_crash` based on the hour (`crash_hour`) that the crash occurred and **save in a new dataset** called `ncbikecrash_time`. If the crash occurred during from 12:00 am to 6:59 am, define `time_crash` to be “Early morning.” If the crash occurred from 7:00 am to 11:59 am, define `time_crash` to be “Morning.” From 12:00 pm to 5:59 pm “Afternoon,” and from 6:00 pm to 11:59 pm “Evening.”

Then, create a table which counts the number of crashes that occurred during these time periods. When do most crashes occur? **Note:** For this question, you should have one section of code for defining the new dataset with the new variable, and a second section of code for creating the table. (They can be in the same code chunk, but they’ll be two separate pieces.)

Exercise 6

- Calculate the mean driver age involved in crashes in all NC counties.
- Create a table which displays the county name and the mean driver age in those counties. In your table, display only the top five counties in terms of oldest drivers, sorted from largest to smallest. **Note:** You may need to use `na.rm = T` to remove NA values.

Exercise 7

(a)

Create a side-by-side boxplot showing the distribution of biker age (`bike_age`) by `region` . Include an informative title. Fill the boxplots with a color of your choice (other than the default). [Click here](#) for a list of colors in R. Provide 1 sentence commenting on the distribution of ages across groups.

(b)

Run the following code to create grouped box plots. (The code should already be in your Quarto template.) Add an informative title and informative axis labels. What do you notice about `Snow, Sleet, Hail, Freezing Rain/ Drizzle` driver ages in Rural vs. Urban areas?

```
ggplot(ncbikecrash,
  mapping = aes(x = rural_urban,
    y = driver_age,
    fill = weather)) +
```

```
geom_boxplot()
```

AI Attestation

If you used AI for this assignment, list your prompts below. Cite any other online resources you used. If you did not use AI for this assignment, you may simply state “I did not use AI for this assignment.”

Submission

As you’ve seen previously, we can **Render** the template into an .html file that can be opened by any web browser. To export it as a .pdf, open the file in your web browser and then print to or save as a .pdf document. Your TAs will show you how if you need help! (There is a way to directly knit to a .pdf file, but it’s quite a bit more involved.)

You will submit the PDF documents for labs and homework to Gradescope as part of your final submission.

To submit your assignment:

- Access Gradescope through the menu on the BIOS 600 Canvas site.
- Click on the assignment, and you’ll be prompted to submit it.
- Mark the pages associated with each exercise. All of the pages of your lab should be associated with at least one question (i.e., should be “checked”).
- Select the first page of your .PDF submission to be associated with the “Formatting” section.
- Ensure all code is visible in the PDF document by indenting your code after commas.

Grading

Component	Points
Ex 1	5
Ex 2	3
Ex 3	3
Ex 4	3
Ex 5	3
Ex 6	4
Ex 7	5

Formatting	3
------------	---

The “Formatting” grade is to assess the document format. This includes having a neatly organized document (no excessive output, warnings/messages when loading packages and/or data) with readable code and your name and the date updated in the YAML.

Acknowledgements

This lab was adapted from an activity created as part of Data Science in a Box, accessible at <https://datasciencebox.org/>. For more info regarding the data, contact Libby Thomas at the UNC Highway Safety Research Center at thomas@hsrc.unc.edu or John Vine-Hodge at the NCDOT Division of Bicycle and Pedestrian Transportation at javinehodge@ncdot.gov.